Code execution

This page is the reference for controlling how *Calepin* runs chunks and displays their output. If you are starting from scratch, read Notebooks first for the basic document structure, runtime import, code chunks, and inline results.

Execution model

Calepin collects executable chunks before Typst renders the document, runs them, writes their results to `.calepin`, and then asks Typst to render with those results available. Each programming language runs in a persistent session for the duration of the document build, so objects created in one chunk are available in later chunks with the same engine.

Use chunk options when you need to change what runs, what is shown, or where the output appears.

Output elsewhere

Sometimes you want to run a chunk in one place but show its result somewhere else. Set `results: "hide"` so the chunk runs without showing anything where it is written, give it a `label`, then print its output later with `#calepin.results`:

````
#calepin.chunk("python", label: "summary", echo: false, results: "hide")[
```python
total = 40 + 2
print(f"The total is {total}.")
```
]

Run live, the chunk above shows nothing where it is defined, and its output appears here on request:

#calepin.results("summary")
````

`#calepin.results("summary")` prints that chunk's full output: text, figures, warnings, everything it would have shown in place. You can put the call before or after the chunk, and you can call it more than once to repeat the output.

When the chunk produces a cross-referenced figure or table (a `fig-`, `tbl-`, or `lst-` label), the `@label` reference points to where the figure is shown: the chunk's own position when it is visible, or the relocation when the chunk is hidden. Printing the same figure in more than one place and then referencing it is ambiguous, so Typst reports an error.

Run live, the hidden chunk below shows nothing where it is defined, and its output is printed on request:

```
The total is 42.
```

Supported languages

Calepin has built-in engines for **Python** and **R**, and built-in diagram engines for **Mermaid**, **Graphviz DOT**, **TikZ**, and **D2**.

Any language with a Jupyter kernel also works: use the kernel name as the block language. Popular examples include **Bash** (`bash`), **Julia** (`julia`), **Octave** (`octave`), **Gnuplot** (`gnuplot`), and **Ruby** (`ruby`). Install kernels as described in the Jupyter install section.

Run `jupyter kernelspec list` to see what is registered. Whatever name appears there can be used as a block language directly:

````
```bash
echo "hello from bash"
```
````

Options

Options can be set in three places: as document-wide defaults, as arguments to one chunk, or as `#|` header lines inside a block.

Document defaults

`#calepin.setup` sets defaults for every chunk in the document:

```
#calepin.setup(echo: true, eval: true, results: "verbatim")
```

Chunk arguments

`#calepin.chunk(...)` overrides options for a single chunk. Pass the body as a fenced block and *Calepin* infers the engine from the fence:

````
#calepin.chunk(echo: false, results: "typst")[
```python
print("#strong[42 in Typst]")
```
]
````

42 in Typst

`#|` header lines

You can also place options at the top of a plain fenced block, one per line, each prefixed with `#|`:

````
```r
#| echo: false
#| fig-align: right
plot(mpg ~ hp, data = mtcars)
```
````

The `#|` form keeps options next to the code. Its downside is outside *Calepin*: compiling the `.typ` file directly with `typst` shows the `#|` lines as text inside the code block, while options passed to `#calepin.chunk(...)` do not.

Options reference

Global and chunk options

These options can be set in `#calepin.setup` as document-wide defaults and overridden per chunk.

Option	Default	Meaning
echo	`true`	Show the chunk's source code in the rendered document.
eval	`true`	Execute the code. When `false`, nothing runs and no output is produced (the source can still be shown via `echo`).
error	`false`	When `true`, capture an execution error and render it as output. When `false`, an error in the chunk aborts the build.
warning	`true`	Include warnings emitted by the engine in the output. When `false`, they are suppressed.
message	`true`	Include informational messages emitted by the engine (for example R's `message()` output). When `false`, they are suppressed.

results	`"render"`	How results are shown: `render` (pretty display of values, images, and tables), `verbatim` (raw output in a code block), `typst` (treat output text as Typst markup and render it), or `hide` (run the code but omit its output).
fig-device-format	`"svg"`	Format for figure files written by the engine: `svg`, `png`, `jpeg` (alias `jpg`), or `pdf`. Diagram engines always emit `svg` regardless of this setting.
fig-device-dpi	`150`	Resolution in dots per inch for raster formats (`png`, `jpeg`). Ignored for vector formats (`svg`, `pdf`).
fig-device-width	`6`	Width of the plotting device, in inches.
fig-device-height	`"auto"`	Height of the plotting device, in inches. `auto` derives it from the width and `fig-device-aspect`.
fig-device-aspect	`0.618`	Height-to-width ratio used when `fig-device-height` is `auto`: device height = `fig-device-width` × `fig-device-aspect`.
fig-width	`"70%"`	Width of the figure as rendered in the document. Accepts a Typst length or ratio (for example `70%` or `12cm`) or `auto`.
fig-height	`"auto"`	Height of the figure as rendered in the document. Accepts a Typst length or `auto`.
fig-align	`"center"`	Horizontal alignment of the figure in the document: `left`, `center`, or `right`.
fig-responsive	`true`	HTML output only: allow the figure to shrink to fit narrow viewports (sets `max-width: 100%`). No effect on paged output.

Chunk-only options

These options are understood only on individual chunks; they are not valid keys in `#calepin.setup`. The caption and layout options live here because they apply to one chunk's specific output.

Option	Default	Meaning
engine	inferred	Force the engine for this chunk instead of inferring it from the fence or surrounding context.
body	from fence	Provide the raw code body directly instead of writing a fenced block.
label	auto	Assign a stable chunk identifier used for cross-references and result lookup.

fig-caption	`none`	Caption text for the figure. When set, the output is wrapped in a numbered `figure` that can be cross-referenced.
fig-cap-location	`"auto"`	Where the caption sits relative to the figure: `top`, `bottom`, or `margin`. `auto` uses Typst's default placement.
fig-alt-text	`none`	Accessibility (alt) text for generated images. Empty when unset.
fig-subcaptions	`none`	Per-panel captions for a multi-image chunk, given as an array of strings (one per image, in order).
fig-layout-columns	`"auto"`	Column layout for a multi-image chunk: an integer number of equal columns, an array of explicit track sizes, or `auto` to choose a count from the number of images.
fig-layout-rows	`"auto"`	Row layout for a multi-image chunk: an integer number of equal rows, an array of explicit track sizes, or `auto`.

Quarto-style names

Chunk options have different names in *Calepin* and Quarto. Some Quarto aliases are accepted, but using them is not recommended, and *Calepin* emits a warning when it meets an unsupported name. The accepted aliases are:

- `out-width` maps to `fig-width`
- `out-height` maps to `fig-height`
- `out-align` maps to `fig-align`
- `fig-alt` maps to `fig-alt-text`
- `fig-subcap` maps to `fig-subcaptions`
- `fig-format` maps to `fig-device-format`
- `fig-dpi` maps to `fig-device-dpi`
- `layout-ncol` maps to `fig-layout-columns`
- `layout-nrow` maps to `fig-layout-rows`